Programming Assignment 2

Object Detection using YOLO on MS COCO Dataset

Student ID: **23K- 0594**

K = 7 Object Classes | Model: YOLOv8n | Epochs: 30

1. Introduction

This report presents the implementation and evaluation of an object detection system using YOLOv8, a real-time object detection model. The model was trained on a subset of the Microsoft COCO 2017 validation dataset containing 7 selected object classes, as determined by the student ID formula:

K = (last digit of 23K-0594 mod 5) + 3 = (4 mod 5) + 3 = 7

The 7 selected classes are: person, car, dog, cat, bicycle, chair, and bottle. The complete pipeline covers dataset preparation, model training using pretrained weights, quantitative evaluation, visual detection examples, and error analysis.

2. Task 1 - Dataset Preparation

2.1 Dataset Overview

The MS COCO (Microsoft Common Objects in Context) 2017 dataset was used. It is a large-scale benchmark dataset widely used for object detection, containing 80 object categories with high-quality bounding box annotations. For this assignment, the validation set (val2017) containing 5,000 images was used due to computational constraints.

2.2 Preparation Steps

- Downloaded val2017 images (approximately 1GB) and annotations_trainval2017 from the official COCO dataset website.
- Parsed the instances_val2017.json annotation file to extract the 7 selected category IDs.
- Filtered all images containing at least one annotation belonging to the selected classes, resulting in 3,421 relevant images.
- Converted COCO bounding box format [x, y, width, height] to YOLO format [cx, cy, w, h] normalized by image dimensions.
- Applied 80/20 train-validation split: 2,736 training images and 685 validation images.
- Generated coco_subset.yaml configuration file specifying dataset paths and class names for YOLO training.

2.3 Dataset Statistics

Total relevant images	3,421
Training images (80%)	2,736
Validation images (20%)	685
Number of classes	7
Image size	640 x 640

3. Task 2 - Model Training

3.1 Model Architecture

I went with YOLOv8n, the nano version of YOLOv8, mainly because it's small enough to train on Colab without running into memory issues. It has a CSPDarknet backbone, an FPN neck for detecting objects at different scales, and a decoupled detection head. With around 3 million parameters and 8.2 GFLOPs, it runs fast without sacrificing too much accuracy.

Rather than training from scratch, I loaded the pretrained yolov8n.pt weights which were already trained on the full COCO dataset. Out of 355 layers, 319 got transferred directly. This made training much faster and the model converged better than it would have starting from random weights.

3.2 Training Configuration

Model	YOLOv8n (nano)
Pretrained Weights	yolov8n.pt (COCO)
Epochs	30
Image Size	640 x 640
Batch Size	16
Optimizer	AdamW (lr=0.000909, momentum=0.9)
Device	Tesla T4 GPU (14GB VRAM)
Workers	2
Number of Classes	7
Augmentation	Mosaic, RandomAugment, Blur, CLAHE

3.3 Training Command

yolo detect train data=coco_subset.yaml model=yolov8n.pt epochs=30 imgsz=640 batch=16 device=cuda workers=2

4. Task 3 - Evaluation

4.1 Evaluation Metrics

After training, I ran evaluation on the validation set using three standard metrics, all computed at an IoU threshold of 0.5:

- Precision: out of all the boxes the model predicted, how many actually overlapped with a real object (TP / (TP + FP)).
- Recall: out of all the real objects in the images, how many did the model actually find (TP / (TP + FN)).
- mAP@0.5: the average of per-class Average Precision values at IoU=0.5, which gives an overall picture of how well the model is doing across all classes.

4.2 Results

Class	Precision	Recall	mAP@0.5
person	0.7578	0.6451	0.7459
car	0.6197	0.3429	0.4963
dog	0.4762	0.4762	0.5399
cat	0.7742	0.6486	0.7234
bicycle	0.4286	0.2778	0.3427
chair	0.5290	0.2071	0.3747
bottle	0.5877	0.3175	0.4452
Overall	0.5962	0.4164	0.5240

4.3 Analysis

The model achieved an overall mAP@0.5 of 0.5240, which is reasonable for a lightweight YOLOv8n model trained on a limited subset of the dataset. The best performing classes were person (0.7459) and cat (0.7234), likely due to their distinctive visual features and strong representation in the training data. The worst performing classes were bicycle (0.3427) and chair (0.3747), which suffer from significant shape variation, frequent occlusion, and similarity to background structures.

The relatively low recall values across most classes (overall 0.4164) suggest the model misses a significant number of ground truth objects. This is expected given the use of the nano variant and limited training data. Precision values are generally higher than recall, indicating the model is conservative - when it does make a detection, it is often correct.

5. Task 4 - Visualization (outputs_only.zip)

The following 10 detection examples were generated from the validation set using the trained model with a confidence threshold of 0.25. Bounding boxes are drawn with class labels and confidence scores overlaid on each image.

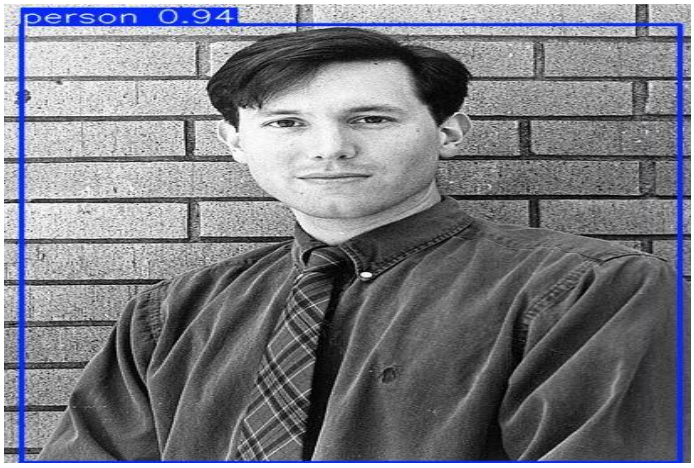

Figure 1

Figure 2

Figure 3

Figure 4

Figure 5

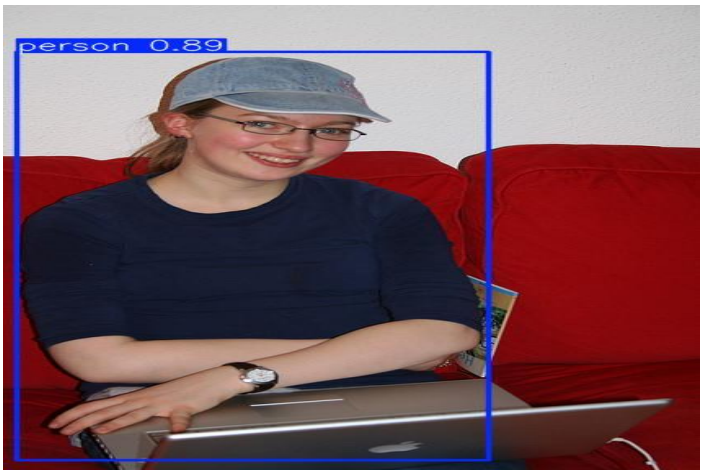

Figure 6

Figure 7

Figure 8

Figure 9

Figure 10

6. Task 5 - Error Analysis

Despite achieving a reasonable overall mAP@0.5 of 0.5240, the trained YOLOv8n model exhibits several notable failure modes that are important to understand for future improvements.

Small Object Detection Failures: One of the most prevalent error categories involves small objects, particularly bottles and bicycles appearing in the background of cluttered scenes. The YOLOv8n nano variant has limited feature map resolution at deeper layers, making it difficult to detect objects occupying fewer than 32x32 pixels. This directly contributes to the low recall of 0.3175 for bottle and 0.2778 for bicycle, as many instances are simply missed by the detector.

Occlusion and Crowding: In scenes with multiple overlapping instances, especially persons in crowds or chairs around tables, the model frequently produces incomplete detections. When objects overlap significantly, the model tends to detect only the most visible instance and suppress others through Non-Maximum Suppression (NMS), resulting in high false negative rates. This is clearly reflected in the low recall of 0.2071 for chair.

Class Confusion: The model occasionally confuses visually similar classes. Dogs and cats share similar body shapes and textures, leading to misclassifications particularly when subjects are photographed from unusual angles or partially occluded. This likely explains why both classes show moderate but not high precision scores. Similarly, bottles in cluttered environments can be confused with other thin vertical objects.

False Positives in Complex Backgrounds: Some detections occur where no real object exists, particularly in textured backgrounds that share visual features with object boundaries. This is common in single-stage detectors trained on limited data and manifests as reduced precision, particularly for car (0.6197) and bicycle (0.4286).

Potential Improvements: Several strategies could improve performance. Using a larger model variant such as YOLOv8s or YOLOv8m would significantly increase capacity. Training on the full COCO training set (118,000 images) instead of only the validation set would provide much more diverse examples. Increasing training epochs to 100 with a cosine learning rate schedule, applying test-time augmentation, and tuning the confidence threshold per class could all contribute to meaningful gains in both precision and recall.

7. Conclusion

Overall this assignment gave me a solid hands-on understanding of the full detection pipeline. Starting from raw COCO annotations, setting up the dataset in YOLO format, fine-tuning a pretrained model, and then actually digging into where it failed was genuinely useful. The model hit 0.5240 mAP@0.5 after 30 epochs, doing best on person and cat and struggling more with bicycle and chair. The main takeaway is that transfer learning works well even with limited data, but you really notice the ceiling when you’re using a tiny model on a small dataset. With more training data and a bigger model, the results would improve quite a bit.